

Algorithm Benchmarking and Outcomes

**Society Connectedness & Health Lab
Yale School of Public Health**

Church Closures Dashboard Project



Table of contents

Generating the Dashboard Map Metrics from the 2023 Format Data	2
Overview	2
Important Considerations	3
Archival of Original Scripts	3
Ignored Files	3
Environment Reproducibility	4
Methods	6
Closures and Reopenings	7
Persistence	7
Scenarios to Consider:	8
Methods and Modifications:	8
Calculate the Rates	9
Rates Per 10,000 Persons	10
Rates Per Square Mile	11
Combined by Geography and Converted to GeoJSON	13
Results	13
Appendix	15
Fill in Minor Years-Open Gaps	15
Generate the Openings, Closures, and Persistence Counts	15
Generate Rates Per 10,000 Persons	16
Generate Rates Per Square Mile	17
Final Results	18
References	19

Generating the Dashboard Map Metrics from the 2023 Format Data

Prepared By: Shelby Golden, M.S. from the Yale School of Public Health [Data Science and Data Equity](#) team

Date Created: August 30th, 2026

Date Updated: August 30th, 2026

Overview

The project objectives were as follows:

1. **REVIEW:** Evaluate the raw data and previously outlined cleaning, validation, and harmonization methods employed by [Dr. Insang Song](#) for accuracy and identify opportunities for improvement.
2. **PREPARATION:** Implement identified improvements to prepare the dataset for analysis and distribution to qualifying collaborators.
3. **METRICS:** Calculate the metrics necessary for dynamic visualization.

Additionally, the project Principal Investigator (PI), [Professor Yusuf Ransome](#), requested the pipeline be reproducible for future collaborators and readily adaptable to potential changes in project scope.

Following receipt of the raw data in Summer 2025, a review was conducted and findings were compared against the original methodology provided by Dr. Song. When an updated dataset was subsequently delivered in May 2026, the pipeline was expanded to accommodate two designated variations: the **2023 Format** and the **2026 Format**.

This document outlines the methodological changes made to ensure metric compatibility with dynamic dashboard map visualization. The first drafts of these algorithms are described alongside an assessment of their performance and areas for improvement. General lessons learned throughout this process, such as strategies for optimizing algorithm speed and accuracy, are also discussed.

Results generated from this pipeline were used for the Summer 2025 dashboard [prototype](#)

developed by [Gordon Tu](#).

Important Considerations

Selected code excerpts are referenced below. The complete code used to generate this report can be found in “[Code/Generate the Metrics_2023 Format.R](#)” with all packages required documented in the **SET UP THE ENVIRONMENT** section. Custom functions used throughout the script are in “[Code/Support Functions/For Generate the Metrics_2023 Format.R](#)”.

Archival of Original Scripts

Given the time constraints of the Summer 2025 prototype launch, algorithm performance and accuracy were not improved in this iteration, and not all 2023 Formatted data could be processed — either due to algorithm limitations or time constraints.

Following the prototype release, the original scripts reflecting the step-wise methods were formalized and published on GitHub in preparation for processing the remaining 2023 Formatted data. However, upon receipt of the 2026 format in Spring 2026, much of the pipeline was overhauled to accommodate this new version, and updating the original scripts was not prioritized.

Any scripts from this pipeline that have been made available are retained for posterity only. Users should refer to the 2026 versions as the most accurate and reproducible implementation of this process.

Those seeking to understand the format-specific changes made to these methods for the 2026 Format should refer to the corresponding “Review” and “Preparation” reports.

Ignored Files

Individual-level files are withheld from GitHub and Git tracking in accordance with the Data Use Agreement (DUA) with the data provider. Many of these are stored in `**/KEEP LOCAL/` directories or are explicitly listed under the “Project-Specific Exclusions” section of the `.gitignore` file.

Most files are available on the SOCAH Lab OneDrive for the project, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the “Church-Closures-Dashboard GitHub LOCAL ONLY Files” directory. This directory will contain `README.md` files specifying the intended use and filepath for each file.

If you do not have access to the SOCAH Lab OneDrive, carefully follow the codebase instructions to establish the repository and replicate results on your own device. Raw data access instructions may vary depending on your institution.

Environment Reproducibility

Results were produced using R (v4.5.2), RStudio (v2026.01.1+403), and Quarto (v1.8.27). The `renv` package (v1.1.4) is included to reproduce the coding environment, capturing all relevant packages and their versions. If issues arise when running the scripts, verify that the environment is initialized and that the correct versions of R, RStudio, and Quarto are being used.

Compatibility

Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

First-time users should initialize the environment to install all required packages and ensure a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/Church-Closures-Dashboard/"
```

2. Launch the project by opening the `*.Rproj` file in RStudio.
3. In the R console, activate the environment by running the following lines of code:

Command-Line Interface

```
renv::restore() # Download packages and their version saved in the  
↪ lockfile.
```

i Note

If you are asked to update packages, say no. `renv` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:

RStudio Console

```
- The library is already synchronized with the lockfile.
```

If you experience any trouble with this step, you might want to confirm that you are using R (v4.5.2) in the RStudio (v2026.01.1+403) and Quarto (v1.8.27). You can also read more about `renv` in their [vignette](#)¹.

Methods

Four metrics were requested for visualization on the dashboard:

1. Number of churches closed
2. Churches closed rate per 10,000 persons
3. Churches closed per square mile
4. Church persistence rate

The first metric represents a simple count of churches closed within a given area. The remaining three were calculated using modified versions of the definitions developed by Dr. Insang Song. Refer to [“code/church_tabulation_5year_periods.R”](#) in his GitHub repository for further details.

The non-rate metrics were calculated first in **SUBSECTION B4: Generate the Openings, Closures, and Persistence Counts**, as the denominator information required for the rate metrics is not introduced until the time of their calculation, after aggregation to the census boundary. The rate metrics are computed in **PART C: CALCULATE THE RATE METRICS**.

Metrics are calculated across all user-selectable date ranges from 2001 to 2021 using the custom function `find_date_combinations()`. The minimum selectable range is five years, as closure event detection requires identifying businesses that have not filed for four or more consecutive years. Applying this constraint to all valid start and end year combinations within the 2001–2021 period yields 136 possible date ranges.

Leading up to the Summer 2025 prototype release, it was discussed which decennial year basemap should be used: should a single basemap apply across all user-selected date ranges? Should it update automatically based on the user’s selection? Should the user be able to manually select a basemap independent of their chosen date range?

`decide_reference()` is a custom function that determines which decennial year basemap to use based on the user’s selected date range, supporting two methods: “ending” selects the basemap corresponding to the final year of the range; “spanning” selects the basemap covering the decennial period into which the majority of the selected years fall.

Interim Methodology

The original approach was to process data with census boundaries appended via the spatial join in **Step 5**. However, this proved more computationally intensive and

time-consuming than expected.

As a result, **Step 4** was used temporarily to develop this script. Census boundaries were approximated by applying the 2010 ZCTA-to-County crosswalk file to annotate ZIP codes with county and state Federal Information Processing Standards (FIPS) codes within the **Step 4** results using the custom function `add_fips_codes()` (see **SUBSECTION A3: Add FIPS Codes**)^{2,3}. The crosswalk file data source is consistent with that used by Dr. Insang Song in his implementation of this process.

Once more results from **Step 5** became available, they were used in place of the **Step 4** results to generate the final dashboard metrics used at the Summer 2025 symposium.

Closures and Reopenings

A closure is defined as an event in which four or more consecutive years have no filings under any address. To simplify metric calculations, these missing years will be filled in. Care was taken to ensure that this process does not induce duplicate records in the event of temporary relocations within intervening years.

A reopening is defined as a run of at least two consecutive 1s (i.e., “11” or “101”) following a closure. A correction was applied such that the reopening count is equal to or no more than one greater than the closure count for that date subset.

Zeros were filled in using the patterns `c("10{1}1", "10{2}1", "10{3}1")` via the custom functions `fill_zeros()` and `fill_zeros_with_progress()`. These resulting data were used for all downstream metric calculations. Closures and reopenings were then calculated using the custom function `count_closures()`. Metrics generated by this process were validated using simulated data under **SUBSECTION B3: Validation**.

Refer to Appendix - [Fill in Minor Years-Open Gaps](#) and [Generate the Openings, Closures, and Persistence Counts](#) for information on where to find the code that generates these results, as well as the results codebook.

Persistence

The definition provided in Dr. Insang Song’s code:

“Persistence, defined as the sum of institutions during the period. A higher value

indicates fewer closures.”

It was calculated using a lookahead, where entries that ended with a 1 OR was open during the date range AND after the date range was determined to be persistent.

Scenarios to Consider:

With the dynamic slider, it is possible to select a range where only zeros were recorded after the end date, despite the church having been persistently open otherwise. This risk is greater for sub-selections closer to the last recorded date, where some entries lacked a response in the final years of the record. These gaps need not necessarily indicate a closure event under our definition of 4 consecutive years without a response.

The user may also select an end date that coincides with the last recorded date. In this scenario, a church may fail the persistence test because no information is available after the end date and the final recorded status was 0, not 1, even if the church was otherwise persistent throughout the remainder of the selected period.

The selected range may also span a temporary closure period, potentially underestimating the degree to which the organization was otherwise open.

Methods and Modifications:

Two methods will be made available for evaluating persistence. The first replicates the “lookahead” approach used in Dr. Song’s published work. The second interprets persistence strictly within the selected range, contextualizing it over the entire available record rather than looking forward only.

When calculating persistence using the “lookahead” method, any trailing one to three zeros are replaced with ones to avoid undercounting persistence near the last recorded date. This modification is applied temporarily and is not retained after persistence has been estimated.

The custom function developed for this purpose is `get_persistence(method = c("ratio", "lookahead"))`, which used `replace_zero_ends()` to replace trailing one to three zeros with ones. Metrics generated by this process were validated using simulated data under **SUBSECTION B3: Validation**.

Refer to [Appendix - Generate the Openings, Closures, and Persistence Counts](#) for information

on where to find the code that generates these results, as well as the results codebook.

RStudio Console

```
# SUBSECTION B4: Generate the Openings, Closures, and Persistence Counts
# -- Individual Line Results -----

# Initialize progress bar
pb = txtProgressBar(min = 0, max = nrow(all_ranges), style = 3)

# Iterate over all possible date ranges.
for(i in 1:nrow(all_ranges)) {
  df <- df %>%
    mutate(count_closures(df, all_ranges$startDate[i],
      ↪ all_ranges$endDate[i]),
      get_persistence(df, all_ranges$startDate[i],
      ↪ all_ranges$endDate[i]))

  # Print the for loop's progress.
  setTxtProgressBar(pb, i)
}
```

Calculate the Rates

In “[Code/Clean Raw Data_Step 5_2023 Format.R](#)”, `tigris` was used to pull the census tract, county, and state boundary information associated with an address, longitude, and latitude for a given decennial year⁴. However, these results do not correctly retrieve the population data sourced from the same package for each decennial period. The previous method has been reviewed and is expected to accurately reflect the available simple features (SF) data.

Alternatively, the `tidycensus` package can be used to retrieve total population values from the US Census Bureau’s decennial census API. During prototyping, however, it was found that values could not be retrieved for the `get_decennial(variables = "P001001")` table across all three decennial periods. To address this, two approaches can be taken when querying for total population values:

1. Set the affected entries to NA.
2. Use the nearest available decennial database instead.

To handle missing county, state, and decennial combinations, both custom functions used to retrieve their respective denominator values — `fetch_population_data()` via `tidycensus` and `fetch_sf_data()` via `tigris` — include fallback support for alternative decennial years, defined as:

RStudio Console

```
alternatives <- data.frame(
  decennial_year = c(2000, 2010, 2020),
  alternative_1   = c(1990, 2000, 2010),
  alternative_2   = c(2010, 2020, 2000)
)
```

The function iterates over the two alternatives in order, and if both are exhausted, the population value is set to NA. The use of alternative fallback databases for total population can be disabled by setting `allow_alternative = FALSE`.

Rates Per 10,000 Persons

The custom function `fetch_population_data()` retrieves population data from the US Census Bureau API via the `tidycensus` package for each unique county-state FIPS combination, or by state alone when the data is state-level⁵. The function supports two query types by setting `use_acs = [TRUE|FALSE]`: decennial survey results `get_decennial(variables = "P001001")` and American Community Survey (ACS) results `get_acs(variables = "B01003_001")`.

Prior to generating the metrics using this method, a preliminary test is run using `fips_combination_exists()`, which summarized the presence or absence of the population table for each unique county-state FIPS combination, or by state alone when the data is state-level, across all three decennial years. State and county FIPS combinations were checked for each decennial year using `get_decennial(variables = "P001001", year = {2000, 2010, 2020})` to confirm their existence in the database. `get_acs(variables = "B01003_001")` was excluded, as the decennial census population value is preferred.

Rates were calculated using the custom function `calculate_closure_rates()` applied to results pre-aggregated to the county or state level:

RStudio Console

```

# SUBSECTION C2: Generate Rate Per 10,000 Persons
# -- Aggregated by County -----

df_10Krate_county <- calculate_closure_rates(
  df_county,
  geography = "county",
  use_acs = FALSE,
  level = "county",
  allow_alternative = TRUE
)

# -- Aggregated by State -----

df_sqMilerate_state <- calculate_closure_rates_per_sq_mile(
  df_state,
  geography = "state",
  use_acs = FALSE,
  level = "state",
  allow_alternative = TRUE
)

```

⚠ Population Data Retrieval Limitations for the 2000 and 2010 Decennial Periods

The population API methods failed to retrieve the appropriate population values for the 2000 and 2010 decennial periods. The alternative source used was not recorded at the time; time constraints precluded revisiting this algorithm prior to the prototype release.

Refer to [Appendix - Generate Rates Per 10,000 Persons](#) for information on where to find the code that generates these results, as well as the results codebook.

Rates Per Square Mile

The custom function `fetch_sf_data()` retrieves land area data from the US Census Bureau API via the `tigris` package for each unique county-state FIPS combination, or by state alone when the data is state-level⁴. The function searches for state-level land area using

`states(cb = TRUE, year = {2000, 2010, 2020})`), filtered by the `STATEFP` variable for the 2020 decennial year and `STATE` for 2000 and 2010. Similarly, county-level land area is retrieved using `counties(state = state_fips, cb = TRUE, year = {2000, 2010, 2020})`), filtered by the `COUNTYFP` variable for all three decennial years.

Prior to generating the metrics using this method, a preliminary test is run using `fips_combination_exists_sf()`, which summarized the presence or absence of the land area table for each unique county-state FIPS combination, or by state alone when the data is state-level, across all three decennial years.

The retrieved land area variable was then first converted to square meters using the `sf` package function `st_area(land_sf)`. Because `tigris` geometries use a geographic Coordinate Reference System (CRS) (North American Datum of 1983 (NAD 83), European Petroleum Survey Group (EPSG):4269) by default, `st_area()` returns values in square meters^{4,6}. These resulting values were then converted to square miles.

Rates were calculated using the custom function `calculate_closure_rates_per_sq_mile()` applied to results pre-aggregated to the county or state level:

RStudio Console

```
# SUBSECTION C3: Generate Rate Per Square Miles
# -- Aggregated by County -----

df_sqMilerate_county <- calculate_closure_rates_per_sq_mile(
  df_county,
  geography = "county",
  use_acs = FALSE,
  level = "county",
  allow_alternative = TRUE
)

# -- Aggregated by State -----

df_10Krate_state <- calculate_closure_rates(
  df_state,
  geography = "state",
  use_acs = FALSE,
  level = "state",
  allow_alternative = TRUE
)
```

)

Refer to [Appendix - Generate Rates Per Square Mile](#) for information on where to find the code that generates these results, as well as the results codebook.

Combined by Geography and Converted to GeoJSON

In **PART D: COMPILE RESULTS AND FORMAT FOR THE DASHBOARD**, each of these separate results was combined into a single table for each aggregated geography level (county and state) and then converted to GeoJSON with the corresponding GEOID polygons joined. For each census geography level, results generated in prior sections were combined and organized using the custom function `reorder_columns()` by metric — closure, reopening, persistence, rate per 10,000 persons, and rate per square mile — for every possible start and end date combination.

`combine_geocoding()` takes a decennial census data frame and attaches geographic boundary shapes to it by downloading the matching Census Bureau shapefiles via `tigris` for each unique census year (2000, 2010, 2020) ⁴. It automatically handles differences in FIPS column naming across years, detects whether the data is state- or county-level, joins the appropriate shapefiles by FIPS code, and returns a single combined map-ready data frame.

The final results were written to GeoJSON format using `st_write()` from the `sf` package ⁶.

Refer to [Appendix - Final Results](#) for information on where to find the code that generates these results, as well as the results codebook.

Results

Loading the relevant denominator and census boundary polygon information at the time of calculation likely contributed to increased computation time when generating these metrics. Additionally, `tidyverse` functions were heavily used for data frame-wide operations. In **PART A: ENHANCING FUNCTION PERFORMANCE AND EFFICIENCY** of “[Clean Raw Data_Step 1_2023 Format.R](#)”, it was found that the `data.table` format and its associated operations were significantly faster and more computationally lightweight when compared with its `tidyverse` counterpart ^{7,8}.

Additionally, limiting the total population table search to `get_decennial(variables = "P001001")` across all decennial periods resulted in failed matches for all entries associated with the 2020 decennial period. Due to the way the function was constructed, the alternative decennial period actually used to match with the US Census Bureau database was not recorded. This issue occurred for both county- and state-level total population queries. All expected land area values matched the expected decennial period. In both cases, no NA values were introduced during the API querying process.

Rather than allowing the query to use an alternative year, further research should be conducted to identify the correct total population lookup table value for the 2020 decennial period, as it appears to have changed after 2010 census.

Appendix

Fill in Minor Years-Open Gaps

For the implementation, refer to **SUBSECTION A2: Fill in Minor Years-Open Gaps** in “Code/Generate the Metrics_2023 Format.R”. All available **Step 5** results were processed, with results written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/Summer 2025 Dashboard Prototype_ARCHIVED/Confirm No Duplicates Added After Filling Ones_05.26.2025.csv`.

Table 1: Codebook for Evaluation Output Fields

Field	Description
abi	Unique business identifier. Evaluation is performed over each unique business ID.
decennial_census	The decennial census period: 2000, 2010, or 2020. Each value should be replicated across all rows, as rows represent distinct census boundary metadata records.
2001:2021	Column-wise sum of all entries associated with the given business ID.
all_counts_0_or_1	Boolean. TRUE if all date entry sums for the given business ID are equal to 0 or 1.

Generate the Openings, Closures, and Persistence Counts

For the implementation, refer to **SUBSECTION B4: Generate the Openings, Closures, and Persistence Counts** in “Code/Generate the Metrics_2023 Format.R”. All available **Step 5** results were processed. Individual line results written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/Summer 2025 Dashboard Prototype_ARCHIVED/Calculate Counts and Persistence_05.29.2025.csv`; aggregations to the county-level census boundary were written to `~/Counts and Persistence By County_05.29.2025.csv`; aggregations to state-level census boundary were written to `~/Counts and Persistence By State_05.29.2025.csv`.

Table 2: Codebook for New Evaluation Output Fields

Field	Description
<code>closure_count_YYYY-YYYY</code>	Integer. The count of closures observed between the specified start and end years.
<code>reopening_count_YYYY-YYYY</code>	Integer. The count of reopenings observed between the specified start and end years.
<code>persistence_YYYY-YYYY</code>	Integer. A binary indicator of whether persistence was observed between the specified start and end years, where 1 indicates persistence and 0 indicates none.

NOTE: The suffix YYYY-YYYY represents the start and end years of possible date range selections (**SUBSECTION A1: Define the Date Ranges**), where each year is a value between 2001 and 2021 (e.g., 2004–2017).

Generate Rates Per 10,000 Persons

For the implementation, refer to **SUBSECTION C2: Generate Rate Per 10,000 Persons** in “[Code/Generate the Metrics_2023 Format.R](#)”. All available **Step 5** results, aggregated to the county or state level in **SUBSECTION B4: Generate the Openings, Closures, and Persistence Counts** were processed. Aggregations to the county-level census boundary were written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/Summer 2025 Dashboard Prototype_ARCHIVED/10K Rate By County_05.30.2025.csv`; aggregations to state-level census boundary were written to `~/10K Rate By State_05.30.2025.csv`.

Table 3: Codebook for New Evaluation Output Fields

Field	Description
<code>population</code>	Population value for the given county and decennial year combination. The decennial year used depends on whether the API call found a match in the tidycensus database; if not, an alternative decennial year may have been used.
<code>alternative_used</code>	Boolean. TRUE if an alternative decennial year was used to source the population count, FALSE otherwise.

Continued on next page

Field	Description
<code>closure_rate_per_10000_YYYY-YYYY</code>	Closure rate calculated by dividing the closure count by population, scaled per 10,000.

NOTE: The suffix YYYY-YYYY represents the start and end years of possible date range selections (**SUBSECTION A1: Define the Date Ranges**), where each year is a value between 2001 and 2021 (e.g., 2004–2017).

Generate Rates Per Square Mile

For the implementation, refer to **SUBSECTION C3: Generate Rate Per Square Miles** in “[Code/Generate the Metrics_2023 Format.R](#)”. All available **Step 5** results, aggregated to the county or state level in **SUBSECTION B4: Generate the Openings, Closures, and Persistence Counts** were processed. Aggregations to the county-level census boundary were written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/Summer 2025 Dashboard Prototype_ARCHIVED/Sq Mile Rate By County_05.30.2025.csv`; aggregations to state-level census boundary were written to `~/Sq Mile Rate By State_05.30.2025.csv`.

Table 4: Codebook for New Evaluation Output Fields

Field	Description
<code>land_area</code>	Land area of the given county in square miles, retrieved for the given decennial year combination. The decennial year used depends on whether the API call found a match in the tidycensus database; if not, an alternative decennial year may have been used.
<code>alternative_used</code>	Boolean. TRUE if an alternative decennial year was used to source the land area, FALSE otherwise.
<code>closure_rate_per_sq_mile_YYYY-YYYY</code>	Closure rate calculated by dividing the closure count by <code>land_area</code> (square miles).

NOTE: The suffix YYYY-YYYY represents the start and end years of possible date range selections (**SUBSECTION A1: Define the Date Ranges**), where each year is a value between 2001 and 2021 (e.g., 2004–2017).

Final Results

For the implementation, refer to **PART D: COMPILE RESULTS AND FORMAT FOR THE DASHBOARD** in “[Code/Generate the Metrics_2023 Format.R](#)”. All available **Step 5** results, aggregated to the county or state level with rates calculated in the preceding sections. Aggregations to the county-level census boundary were written to “./Data/Results/Dashboard Datasets/All Metrics By County_06.04.2025.geojson”; aggregations to state-level census boundary were written to “~/All Metrics By State_06.04.2025.geojson”.

Table 5: Codebook for Dashboard-Metric Geometry-Joined Output Fields

Field	Description
decennial_census	The decennial census period: 2000, 2010, or 2020. Each value is replicated across all rows, as rows represent distinct census boundary metadata records.
state_fips	State-level Federal Information Processing Standard (FIPS) code representing the census boundary level to which the data was aggregated.
county_fips	County-level Federal Information Processing Standard (FIPS) code representing the census boundary level to which the data was aggregated. This variable is absent if the data were aggregated to a higher geographic level.
geoid	The GEOID for the census boundary level, representing a mutually exclusive boundary designation that may include a combination of state and county FIPS, tract, block group, and block IDs.

Continued on next page

Field	Description
geometry	Multipolygon representation of the GEOID boundary, plotted in the mapped component containing the relevant outcome results.
closure_count_YYYY-YYYY	Integer. The count of closures observed between the specified start and end years.
reopening_count_YYYY-YYYY	Integer. The count of reopenings observed between the specified start and end years.
persistence_YYYY-YYYY	Integer. A binary indicator of whether persistence was observed between the specified start and end years, where 1 indicates persistence and 0 indicates none.
closure_rate_per_10000_YYYY-YYYY	Closure rate calculated by dividing the closure count by population, scaled per 10,000.
closure_rate_per_sq_mile_YYYY-YYYY	Closure rate calculated by dividing the closure count by land_area (square miles).

NOTE: The suffix YYYY-YYYY represents the start and end years of possible date range selections (**SUBSECTION A1: Define the Date Ranges**), where each year is a value between 2001 and 2021 (e.g., 2004–2017).

References

1. Ushey, K., Wickham, H. & Posit. [Introduction to renv](#).
2. U.S. Census Bureau. 2010 relationship files. Census.gov. <https://www.census.gov/geographies/reference-files/time-series/geo/relationship-files.html> (2024).
3. U.S. Census Bureau. [Explanation of the 2010 ZIP code tabulation area \(ZCTA\) to county relationship file](#). (2020).
4. Walker, K. & Rudis, B. [Tigris: Load census TIGER/line shapefiles](#). (2025).
5. Kyle Walker & Matt Herman. Tidycensus: Load US census boundary and attribute data as 'tidyverse' and 'sf'-ready data frames. <https://walker-data.com/tidycensus/index.html> (2026).

6. Edzer Pebesma et al. Simple features for r. <https://r-spatial.github.io/sf/> (2018) doi:10.32614/RJ-2018-009.
7. Wickham, H. & RStudio. [Tidyverse: Easily install and load the 'tidyverse'](#). (2023).
8. Barrett, T. et al. [Data.table: Extension of 'data.frame'](#). (2026).